Accessing the Animation Area

__Accessing the Animation Area

SimTalk provides the *attributes* and *methods* listed in the table of contents to the left for accessing the **Animation Area** of objects with matrix loading space in *3D*.

You can view and access the **attributes and methods** for setting the animation area by clicking **Auto Complete** on the **Edit** ribbon tab of the *Method Editor*.

> **Note**
>
> The settings for the animation area only apply to objects with matrix loading space (*ParallelStation*, *Store*, *Transporter*, and *Container*), for the *PlaceBuffer*, and for the *Worker*. As a rule, you can use the pre-defined settings, unless they do not meet your requirements.

See also

Edit 3D Properties [dialog] > **MU Animation [tab]** > **Animation Area [described]**

Animation Area [described]

Auto Complete

_3D.convertMUAnimationAreaToPaths [SimTalk]

Converts the animation area of the object designated by <Path> to an animation path with storage places.

Type

Method

Syntax

```
<Path>._3D.convertMUAnimationAreaToPaths
```

Example

```
Store._3D.convertMUAnimationAreaToPaths
```

_3D.createMUAnimationAreaFromPaths [SimTalk]

Creates an animation area for *MUs* from animation paths in the object designated by <Path>.

Type

Method

Syntax

```
<Path>._3D.createMUAnimationAreaFromPaths → boolean
```

Return Value

The return value has the data type *boolean*.

true if enough data was available for the conversion.

Example

```
Store._3D.createMUAnimationAreaFromPaths
```

_3D.MUAnimationAreaAbsoluteCenter [SimTalk]

Sets the center of the animation area of the object designated by <Path> in **absolute** values.

Type

Attribute

Syntax

```
<Path>._3D.MUAnimationAreaAbsoluteCenter:real[3]
```

Assignment Value

You can assign a value of data type *array* containing three values of data type *real*.

The values set the **X** position, the **Y** position, and the **Z** position of the **Center** point of the animation area.

You can specify any negative or positive value.

Example

```
MyParallelstation._3D.MUAnimationAreaAbsoluteCenter := [0.1, 0.1, 0.5]
```

See also

Center [animation area]

_3D.MUAnimationAreaRelativeCenter [SimTalk]

_3D.MUAnimationAreaAbsoluteSize [SimTalk]

Sets the size, i.e., the **Absolute Length** and the **Absolute Width** of the animation area of the object designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.MUAnimationAreaAbsoluteSize:real[2]
```

Assignment Value

You can assign an *array* with two values of data type *real*. You can specify a value between 0 and any size.

The values set the **Absolute Length** and the **Absolute Width** of the animation area.

Example

```
MyParallelstation._3D.MUAnimationAreaAbsoluteSize := [2m,2m]
```

See also

Length [text box] - animation area

Width [text box] - animation area

_3D.MUAnimationAreaRelativeSize [SimTalk]

_3D.MUAnimationAreaEnabled [SimTalk]

Activates (`true`) or deactivates (`false`) the animation area of the object designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.MUAnimationAreaEnabled:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyParallelstation._3D.MUAnimationAreaEnabled := true
```

See also

Animation Area [check box]

Animation Area [described]

_3D.MUAnimationAreaShowMUsAsCuboids [SimTalk]

_3D.MUAnimationAreaMURotation [SimTalk]

Sets the rotation of the *MUs* on the animation area of the object designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.MUAnimationAreaMURotation:real/real[3]
```

Assignment Value

You can specify:

- A *real* number which defines the rotation around the negative z-axis at the position at which you inserted the object.
- An *array* of data type *real* with three values. They define the components of the rotation axis around which the parts will be rotated at the position at which you inserted the object.

Examples

```
MyParallelstation._3D.MUAnimationAreaMURotation := 45 // degrees
```

```
MyParallelstation._3D.MUAnimationAreaMURotation := [15, 1, 1, -1]
```

See also

MU Rotation

_3D.MUAnimationAreaOrientation [SimTalk]

Sets the orientation of the animation area of the object designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.MUAnimationAreaOrientation:string
```

Assignment Value

You can assign a value of data type *string*.

You can specify `"XY-plane"`, `"XZ-plane"`, or `"YZ-plane"`.

Example

```
MyParallelstation._3D.MUAnimationAreaOrientation := "XY-plane"
```

See also

Orientation [animation area]

_3D.MUAnimationAreaRelativeCenter [SimTalk]

Sets the center of the animation area of the object designated by <Path> in **relative** values.

Type

Attribute

Syntax

```
<Path>._3D.MUAnimationAreaRelativeCenter:real[3]
```

Assignment Value

You can assign a value of data type *array* containing three values of data type *real*.

The values set the **X** position, the **Y** position, and the **Z** position of the **Center** point of the animation area.

You can specify any negative or positive value.

Example

```
MyParallelstation._3D.MUAnimationAreaRelativeCenter := [0.1, 0.1, 0.5]
```

See also

Center [animation area]

_3D.MUAnimationAreaAbsoluteCenter [SimTalk]

_3D.MUAnimationAreaRelativeSize [SimTalk]

Sets the size, i.e., the **Relative Length** and the **Relative Width** of the animation area of the object designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.MUAnimationAreaRelativeSize:real[2]
```

Assignment Value

You can assign an *array* with two values of data type *real*. You can specify a value between 0 and any size.

The values set the **Relative Length** and the **Relative Width** of the animation area.

Example

```
MyParallelstation._3D.MUAnimationAreaRelativeSize := [1,1]
```

See also

Length [text box] - animation area

Width [text box] - animation area

_3D.MUAnimationAreaAbsoluteSize [SimTalk]

_3D.MUAnimationAreaShowMUsAsCuboids [SimTalk]

Sets if the *MUs* will be animated on the animation area of the object designated by <Path> with cuboids (`true`) or with the graphic of the *MU* as such (`false`).

Remarks

Animating the cuboids is considerably faster than animating the graphics of the *MUs*.

Type

Attribute

Syntax

```
<Path>._3D.MUAnimationAreaShowMUsAsCuboids:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyStore._3D.MUAnimationAreaShowMUsAsCuboids := true
```

See also

Show MUs as Cuboids [check box]

_3D.MUAnimationAreaEnabled [SimTalk]

Accessing MU Animations

__Accessing MU Animations

Plant Simulation provides a set of functions for **MU Animations**.

Remarks

All objects, which can accept parts (*MUs*) or *Workers*, provide **MU Animations**:

- All material flow objects except *Connector* and *Interface*
- *Pure animation objects*, i.e. object which you can address with the method *_3D.getObject*

- *Frames*

You can buffer the entirety or all animations of a type in a value of data type *any*, for example:

```
var a : any := .Materialflow.PickAndPlace._3D.MUAnimations
```

SimTalk provides the *attributes* and *methods* listed in the table of contents to the left for **MU Animations**.

Functions for the Animation Area

In addition, *Plant Simulation* provides a set of functions for the **Animation Area** of objects with a matrix loading space (*ParallelStation*, *Store*, *Transporter* and *Container*), for the *PlaceBuffer* and for the *Worker*:

_3D.MUAnimationAreaAbsoluteSize [SimTalk]

_3D.MUAnimationAreaEnabled [SimTalk]

_3D.MUAnimationAreaMURotation [SimTalk]

_3D.MUAnimationAreaOrientation [SimTalk]

_3D.MUAnimationAreaAbsoluteCenter [SimTalk]

_3D.convertMUAnimationAreaToPaths [SimTalk]

_3D.createMUAnimationAreaFromPaths [SimTalk]

See also

Edit 3D Properties [dialog] > **MU Animation [tab]** > **Animation Area [described]**

_3D.getObject [SimTalk]

_3D.AnimationObject [SimTalk]

Sets the alternative animation object of the object designated by <Path>.

Remarks

_3D.AnimationObject contains an empty string "" by default, meaning that the object itself is the animation object and transports the *MU* itself.

The exception is the *PickAndPlace robot*. Its attribute contains the animation axes which the respective *PickAndPlace robot* has.

Type

Attribute

Syntax

```
<Path>._3D.AnimationObject:string
```

Assignment Value

You can assign a value of data type *string*.

Example

```
MyRobot._3D.AnimationObject := "upperarm.forearm"
```

See also

Edit 3D Properties [dialog] > **MU Animation [tab]** > **Animation Object**

Inherit Graphics

_3D.InheritGraphics [SimTalk]

_3D.MUAnimations.<AnimationPathName>.delete [SimTalk]

Deletes the designated **MU Animation** of the object designated by <Path>.

Remarks

You cannot delete generated animations.

Type

Method

Syntax

```
<Path>._3D.MUAnimations.<AnimationPathName>.delete → boolean
<Path>._3D.MUAnimations.getAnimation(<...>).delete → boolean
```

Return Value

The return value has the data type *boolean*.

true if the animation was deleted successfully.

false if the animation was not deleted.

Example

```
.MaterialFlow.Station._3D.MUAnimations.Default.delete
// deletes the MU animation named 'Default' of the class of the Station
```

_3D.MUAnimations.<AnimationPathName>.getMUAnimationPosition [SimTalk]

Returns the anticipated position of a *MU* on the designated position within this **MU Animation** of the object designated by <Path>.

Type

Method

Syntax

```
<Path>._3D.MUAnimations.<AnimationPathName>.getMUAnimationPosition(RelPos/
AbsPos:real/length) -> length[3]
```

Parameter

The parameter *RelPos*/*AbsPos* is a value of data type *real* or *length*.

If the value has the data type *real*, it designates the **Relative Position** of the coordinate to be computed. The relative position can be a value between 0 and 1. `0` designates the start of the animation and `1` the end of the animation, i. e., the value 100 %.

If the value has the data type *length*, it designates the **Absolute Position** on the animation as the distance starting at the beginning of the animation.

Return Value

The return value is an *array* of data type *length* with three values.

Examples

```
var p := Station._3D.MUAnimations.Default.getMUAnimationPosition(1)
```

```
var p := Station._3D.MUAnimations.Default.getMUAnimationPosition(0.5m)
```

See also

_3D.getMUAnimationPosition [SimTalk]

_3D.MUAnimations.<AnimationPathName>.getTable [SimTalk]

Formats the passed table and then writes the animation data of all saved animations of the designated **MU Animation** of the object designated by <Path> into this table.

Type

Method

Syntax

```
<Path>._3D.MUAnimations.<AnimationPathName>.getTable(Target:table)
<Path>._3D.MUAnimations.getAnimation(<...>).getTable(Target:table)
```

Parameter

The parameter *Target* of data type *table* contains the data of each saved animation.

Example

```
// mua stands for MU animation
var mua:= Buffer._3D.MUAnimations
var created : boolean := mua.createAnimationLine("Animation2")
if created // The animation line was created.
   var t: table // Copy the ‘Default’ line values to the table.
   mua.Default.getTable(t)
   mua.Animation2.setTable(t)
end
```

See also

Edit 3D Properties [dialog] > **MU Animation [tab]**

_3D.MUAnimations.getAnimation [SimTalk]

_3D.MUAnimations.getTable [SimTalk]

_3D.MUAnimations.setTable [SimTalk]

_3D.MUAnimations.<AnimationPathName>.setTable [SimTalk]

_3D.MUAnimations.<AnimationPathName>.PathAnchorPoint [SimTalk]

_3D.MUAnimations.<AnimationPathName>.IsCurve [SimTalk]

Returns if this **MU Animation** of the object designated by <Path> is of type **Polycurve** (true) or not (false).

Type

Read-only attribute

Syntax

```
<Path>._3D.MUAnimations.<AnimationPathName>.IsCurve → boolean
<Path>._3D.MUAnimations.getAnimation(<...>).IsCurve → boolean
```

Return Value

The return value has the data type *boolean*.

Example

```
var b : boolean := .MaterialFlow.Station._3D.MUAnimations.Default.IsCurve
```

See also

_3D.MUAnimations.getAnimation [SimTalk]

_3D.MUAnimations.<AnimationPathName>.IsGenerated [SimTalk]

Returns if this **MU Animation** of the object designated by <Path> is automatically generated (true) or not (false).

Remarks

If you create an animation with the same name, it will take preference.

Type

Read-only attribute

Syntax

```
<Path>._3D.MUAnimations.<AnimationPathName>.IsGenerated → boolean
<Path>._3D.MUAnimations.getAnimation(<...>).IsGenerated → boolean
```

Return Value

The return value has the data type *boolean*.

Example

```
var b : boolean := .MaterialFlow.Station._3D.MUAnimations.Default.IsCurve
```

See also

_3D.CameraAnimations.<AnimationPathName>.setTable [SimTalk]

_3D.MUAnimations.getAnimation [SimTalk]

_3D.MUAnimations.<AnimationPathName>.IsLine [SimTalk]

Returns if this **MU Animation** of the object designated by <Path> is of type **Lines** (true) or not (false).

Type

Read-only attribute

Syntax

```
<Path>._3D.MUAnimations.<AnimationPathName>.IsLine → boolean
<Path>._3D.MUAnimations.getAnimation(<...>).IsLine → boolean
```

Return Value

The return value has the data type *boolean*.

Example

```
var b : boolean := .MaterialFlow.Station._3D.MUAnimations.Default.IsLine
```

See also

_3D.MUAnimations.getAnimation [SimTalk]

_3D.MUAnimations.<AnimationPathName>.IsPoint [SimTalk]

Returns if this **MU Animation** of the object designated by <Path> is of type **Lines**, but only consists of a single anchor point (true) or not (false).

Type

Read-only attribute

Syntax

```
<Path>._3D.MUAnimations.<AnimationPathName>.IsPoint → boolean
<Path>._3D.MUAnimations.getAnimation(<...>).IsPoint → boolean
```

Return Value

The return value has the data type *boolean*.

Example

```
var b : boolean := .Station._3D.MUAnimations.Default.IsPoint
```

See also

_3D.MUAnimations.createAnimationPoint [SimTalk]

_3D.MUAnimations.getAnimation [SimTalk]

_3D.MUAnimations.<AnimationPathName>.IsSpline [SimTalk]

Returns if this **MU Animation** of the object designated by <Path> is of type **Spline** (true) or not (false).

Type

Read-only attribute

Syntax

```
<Path>._3D.MUAnimations.<AnimationPathName>.IsSpline → boolean
<Path>._3D.MUAnimations.getAnimation(<...>).IsSpline → boolean
```

Return Value

The return value has the data type *boolean*.

Example

```
var b : boolean := .MaterialFlow.Station._3D.MUAnimations.Default.IsSpline
```

See also

_3D.MUAnimations.getAnimation [SimTalk]

_3D.MUAnimations.<AnimationPathName>.Length [SimTalk]

Returns the physical length of the **MU Animation** of the object designated by <Path>.

Type

Read-only attribute

Syntax

```
<Path>._3D.MUAnimations.<AnimationPathName>.Length → length
<Path>._3D.MUAnimations.getAnimation(<...>).Length → length
```

Return Value

The return value has the data type *length*.

Example

```
var l : length := Conveyor._3D.MUAnimations.getAnimation(1).Length
```

See also

_3D.MUAnimations.<AnimationPathName>.getMUAnimationPosition [SimTalk]

_3D.MUAnimations.getAnimation [SimTalk]

_3D.MUAnimations.<AnimationPathName>.PathAnchorPoint [SimTalk]

Sets this **MU Animation** of the object designated by <Path> to a single point with the given values of data type *real*.

Type

Attribute

Syntax

```
<Path>_3D.MUAnimations.<AnimationPathName>.PathAnchorPoint:void/real[3/7]
```

Assignment Value

You can assign an *array* of data type *real* with three or seven values.

- Specify an *array* containing three values of data type *real* to set the position of the point while setting the **Rotation Angle** to 0 and the **Axis** to [0,0,-1].
- Specify an *array* containing seven values of data type *real* to designate the **Position** with the first three values, the **Rotation Angle** with the forth value, and **Rotation Axis** with the last three values.
- If you get the attribute, *Plant Simulation* returns an *array* of data type *real* with the seven values of the single path anchor point as described above.

  Plant Simulation returns void if the animation path is not of type **Lines** or has more than one anchor point.

Examples

```
if MyStation._3D.MUAnimations.Default.PathAnchorPoint[3] /= 2m
then MyStation._3D.MUAnimations.Default.PathAnchorPoint := [0,0,2,90,0,0,-1]
```

```
MyStation._3D.MUAnimations.Default.PathAnchorPoint[3] := 2m
```

```
print MyParallelStation._3D.MUAnimations.getAnimation(4,7).PathAnchorPoint
```

See also

_3D.MUAnimations.<AnimationPathName>.getTable [SimTalk]

_3D.MUAnimations.<AnimationPathName>.setTable [SimTalk]

Edit Values [button] - anchor point, MU animation

_3D.MUAnimations.<AnimationPathName>.setTable [SimTalk]

Overwrites this **MU Animation** of the object designated by <Path> with the data of the passed table.

Remarks

The table has to be formatted correctly, otherwise *Plant Simulation* cancels your changes. You can query the format of the table with the method *_3D.MUAnimations.<AnimationPathName>.getTable*.

> **Note**
> Some objects have generated animations. An example is the **MU Animation Path** named **Default** of the *Conveyor*. These animations cannot be overwritten.

The animations of the *table*, which the parameter *Source* contain, do overwrite the previously existing saved animations.

Type

Method

Syntax

```
<Path>._3D.MUAnimations.<AnimationPathName>.setTable(Source:table)
<Path>._3D.MUAnimations.getAnimation(<...>).setTable(Source:table)
```

Parameter

The parameter *Source* of data type *table* contains the new saved animation.

Example

```
var mua := Buffer._3D.MUAnimations
var created : boolean := mua.createAnimationLine("Animation2")
if created // The animation line was created.
   var t: table // Copy the 'Default' line values to the table.
   mua.Default.getTable(t)
   mua.Animation2.setTable(t)
end
```

See also

Edit 3D Properties [dialog] > **MU Animation [tab]**

_3D.MUAnimations.<AnimationPathName>.getTable [SimTalk]

_3D.MUAnimations.getTable [SimTalk]

_3D.MUAnimations.<AnimationPathName>.IsGenerated [SimTalk]

_3D.MUAnimations.<AnimationPathName>.PathAnchorPoint [SimTalk]

_3D.MUAnimations.getAnimation [SimTalk]

_3D.MUAnimations.Count [SimTalk]

Returns the number of the existing animation paths saved in the **MU Animation** of the object designated by <Path>.

Type

Read-only attribute

Syntax

```
<Path>._3D.MUAnimations.Count → integer
```

Return Value

The return value has the data type *integer*.

Example

```
var AniIF : integer := self.~._3D.MUAnimations.Count
   print AniIF.getAnimation(i).Name // prints the names of all saved
animations
next
```

_3D.MUAnimations.createAnimationCurve [SimTalk]

Creates a new saved animation path of type **Polycurve** of the object designated by <Path>.

Remarks

Set the values of the animation path with the method *_3D.MUAnimations.<AnimationPathName>.getTable*.

Type

Method

Syntax

```
<Path>._3D.MUAnimations.createAnimationCurve(Name:string) → boolean
```

Parameter

The parameter *Name* of data type *string* designates the name of the new animation path to be created.

Return Value

The return value has the data type *boolean*.

true if the animation path was created successfully, i.e., if the name has not been assigned yet.

Example

```
var a: boolean := self.~._3D.MUAnimations.createAnimationCurve("Curved
Movement")
if not a then
   print "The animation curve could not be created."
end
```

See also

Edit 3D Properties [dialog] > **MU Animation [tab]**

_3D.MUAnimations.getTable [SimTalk]

_3D.MUAnimations.setTable [SimTalk]

_3D.MUAnimations.createAnimationLine [SimTalk]

_3D.MUAnimations.createAnimationSpline [SimTalk]

_3D.MUAnimations.createAnimationLine [SimTalk]

Creates a new saved animation path of type **Lines** of the object designated by <Path>.

Remarks

Set the values of the animation path with the method *_3D.MUAnimations.setTable*.

Type

Method

Syntax

```
<Path>._3D.MUAnimations.createAnimationLine(Name:string) → boolean
```

Parameter

The parameter *Name* of data type *string* designates the name of the new animation path to be created.

Return Value

The return value has the data type *boolean*.

true if the animation path was created successfully, i.e., if the name has not been assigned yet.

Example

```
var mua := Buffer._3D.MUAnimations("Animation2")
```

See also

Edit 3D Properties [dialog] > MU Animation [tab]

_3D.MUAnimations.getTable [SimTalk]

_3D.MUAnimations.setTable [SimTalk]

_3D.MUAnimations.<AnimationPathName>.setTable [SimTalk]

_3D.MUAnimations.createAnimationSpline [SimTalk]

_3D.MUAnimations.createAnimationPoint [SimTalk]

Creates an animation point for the designated index with the designated position for the object designated by <Path>.

Remarks

This is a new animation path of type **Lines** of the object designated by <Path> consisting of a single point.

Type

Method

Syntax

```
<Path>._3D.MUAnimations.createAnimationPoint(IndexX:integer,
IndexY:integer, Position:length[3]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *IndexX* of data type integer designates the x-position of the animation point.
- The parameter *IndexY* of data type integer designates the y-position of the animation point.
- The parameter *Position* is an *array* of data type *length* containing three values, which designate the position in space within the object coordinate system.

Return Value

The return value has the data type *boolean*.

true if the animation point was created successfully, i.e., if the name has not been assigned yet.

Example

```
.MaterialFlow.ParallelStation._3D.MUAnimations.createAnimationPoint(2, 3,
[2.0, 3.0, 2.0])
```

See also

Edit 3D Properties [dialog] > **MU Animation [tab]**

_3D.MUAnimations.createAnimationLine [SimTalk]

_3D.MUAnimations.getAnimation [SimTalk]

_3D.MUAnimations.createAnimationSpline [SimTalk]

Creates a new saved animation path of type **Spline** of the object designated by <Path>.

Remarks

Set the values of the animation path with the method *_3D.MUAnimations.setTable*.

Type

Method

Syntax

```
<Path>._3D.MUAnimations.createAnimationSpline(Name:string) → boolean
```

Parameter

The parameter *Name* of data type *string* designates the name of the new animation path to be created.

Return Value

The return value has the data type *boolean*.

true if the animation path was created successfully, i.e., if the name has not been assigned yet.

Example

```
var a: boolean := self.~._3D.MUAnimations.createAnimationSpline("spline
movement")
if not a
   print "The animation spline could not be created."
end
```

See also

Edit 3D Properties [dialog] >MU Animation [tab]

_3D.MUAnimations.getTable [SimTalk]

_3D.MUAnimations.setTable [SimTalk]

_3D.MUAnimations.createAnimationCurve [SimTalk]

_3D.MUAnimations.createAnimationLine [SimTalk]

_3D.MUAnimations.getAnimation [SimTalk]

Returns the specified **MU Animation** of the object designated by <Path>.

Type

Method

Syntax

```
<Path>._3D.MUAnimations.getAnimation(AnimationPathName:string)
<Path>._3D.MUAnimations.getAnimation(Index:integer)
<Path>._3D.MUAnimations.getAnimation(PositionX:integer, PositionY:integer)
```

Parameters

The parameters designate a possibility each to query the designated animation.

- The parameter *Name* of data type *string* designates a saved animation, whose name is an illegal *SimTalk* identifier, `"#0#0"` in the example below.
- The parameter *Index* of data type *integer* designates the n-th animation of the animations.
- The parameter set *PositionX* and *PositionY* designates the dimension of the animation of the animation point.
  - The parameter *PositionX* of data type *integer* designates the x-dimension of the animations of the passed indexed animation point, i.e., of the paths with the notation **"#x#y"**, which, for example, designate the individual processing stations of a *ParallelStation* or the storage places of a *Store*.
  - The parameter *PositionY* of data type *integer* designates the y-dimension of the animations of the passed indexed animation point.

    Note

    The indexes, which you enter, start at 1, while the indexes of the path names, on which they are based, start at 0. `getAnimation(1, 1)` corresponds to `getAnimation("#0#0")`.

Return Value

Void if either a *string*, or for *_3D.MUAnimations.getAnimation* two integer values designating a point on the loading space, are passed and if the animation with the specified name or the specified index pair does not exist. This is instead of opening the *Debugger*.

This does not apply to all other error cases, for example if you only specify a single *integer* value, the direct path extension, or specifying unsuited data types.

Examples

```
var t: table
self.~._3D.MUAnimations.getAnimation("#0#0").getTable(t)
```

```
var t: table
self.~._3D.MUAnimations.getAnimation(7).getTable(t)
```

See also

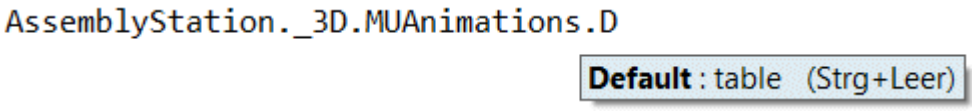
_3D.CameraAnimations.getTable [SimTalk]

_3D.MUAnimations.setTable [SimTalk]

Edit 3D Properties [dialog] > **MU Animation [tab]**

Accessing an Animation via the Path Extension

If a saved animation has a name that is a legal *SimTalk* identifier, you can access this animation via the path extension. This applies, for example, to the **Path Name** > **Default** of the **MU animation**. Here you could, for example, type in `Assembly._3D.MUAnimations.D`. The path extension then suggests



_3D.MUAnimations.getTable [SimTalk]

Queries the animation data of all saved animations of the designated **MU Animation** of the object designated by <Path> and writes it into the specified table.

Remarks

Plant Simulation automatically formats the passed *table*.

Type

Method

Syntax

```
<Path>._3D.MUAnimations.getTable(Target:table)
```

Parameter

The parameter *Target* of data type *table* contains all available saved animations of the designated animation type.

Example

```
var t : table
self.~._3D.MUAnimations.getTable(t)
debug
self.~._3D.MUAnimations.setTable(t)
```

See also

Edit 3D Properties [dialog] > **MU Animation [tab]**

_3D.MUAnimations.setTable [SimTalk]

_3D.CameraAnimations.<AnimationPathName>.getTable [SimTalk]

_3D.MUAnimations.<AnimationPathName>.setTable [SimTalk]

_3D.MUAnimations.setTable [SimTalk]

Overwrites the saved animations of the designated **MU Animation** of the object designated by <Path> with the data of the passed table.

Remarks

The table has to be formatted correctly, otherwise *Plant Simulation* cancels your changes. You can get the format of the table with the method *_3D.MUAnimations.getTable*.

> **Note**
> Some objects have generated animations. An example is the **MU Animation Path** named **Default** of the *Conveyor*. These animations cannot be overwritten.

The animations of the *table*, which the parameter *Source* contain, do overwrite the other previously existing saved animations.

Type

Method

Syntax

```
<Path>._3D.MUAnimations.setTable(Source:table)
```

Parameter

The parameter *Source* of data type *table* contains the new animations.

Example

```
var t: table
self.~._3D.MUAnimations.getTable(t)
debug
self.~._3D.MUAnimations.setTable(t)
```

See also

Edit 3D Properties [dialog] > **MU Animation [tab]**

_3D.MUAnimations.setTable [SimTalk]

_3D.CameraAnimations.<AnimationPathName>.getTable [SimTalk]

_3D.MUAnimations.<AnimationPathName>.setTable [SimTalk]

_3D.MUSideToAttach [SimTalk]

Sets the side on which the *MU* will be attached to the object designated by <Path>.

Remarks

Plant Simulation computes the size of the *MUs* according to the size you set in the **simulation properties**.

Type

Attribute

Syntax

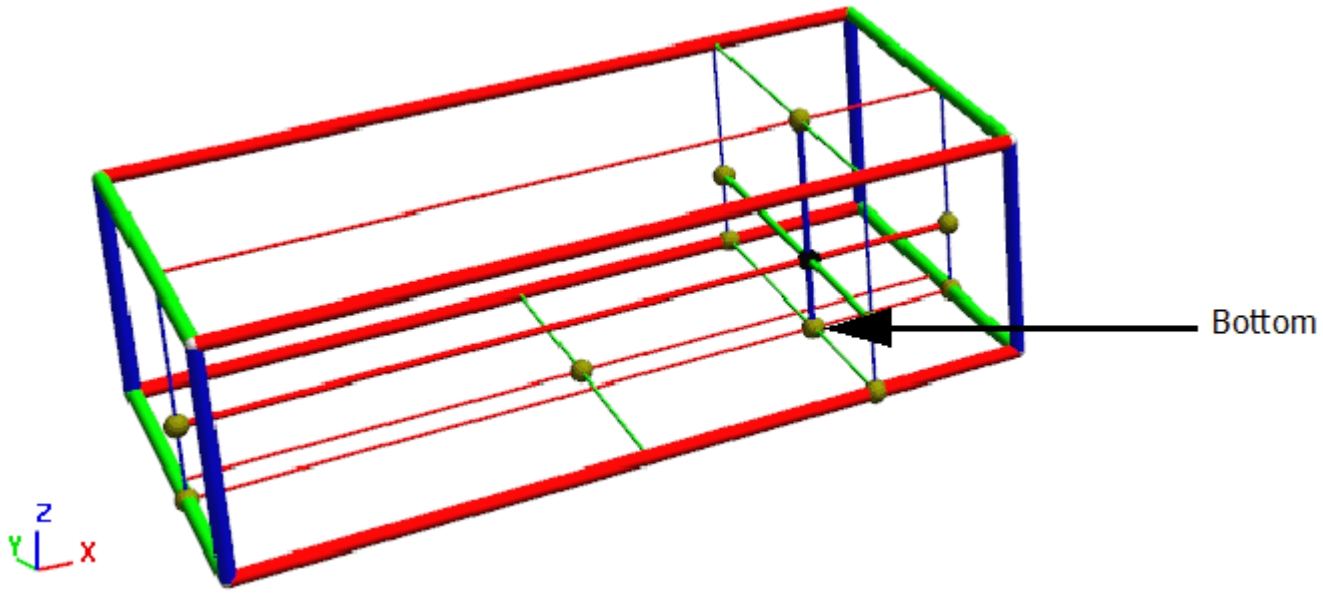
```
<Path>._3D.MUSideToAttach:string
```

Assignment Value

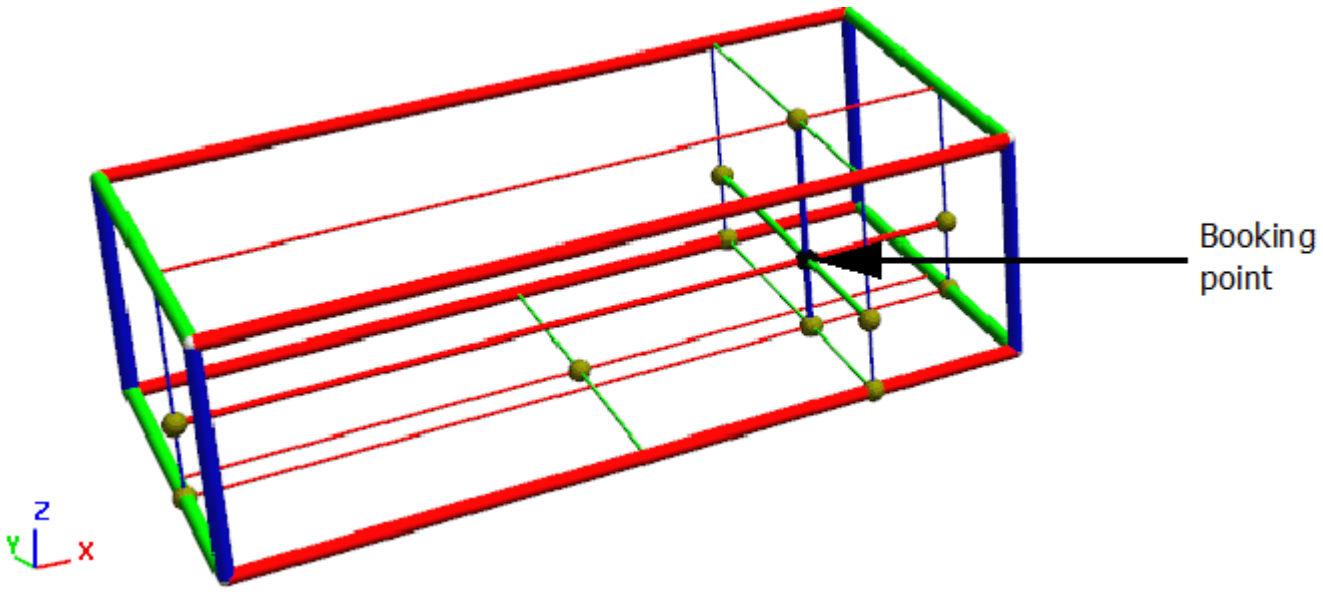
You can assign a value of data type *string*.

Specify the side at which the loaded part is going to be attached to the object.

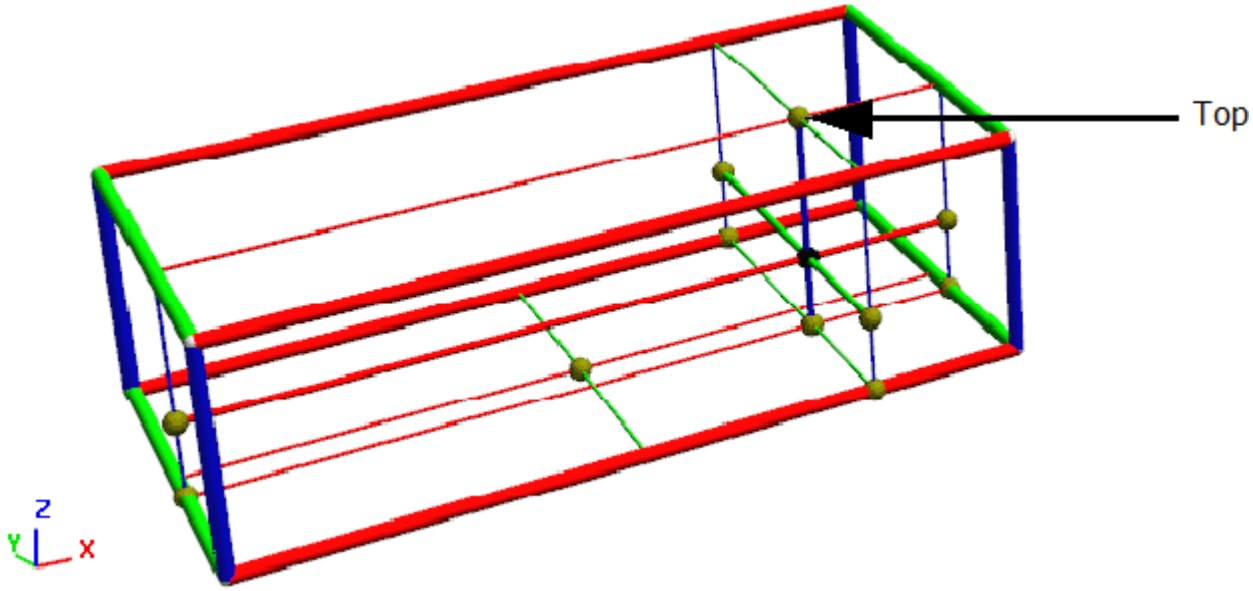
- `"Bottom"`: Attaches the loaded *MU* to the object from the bottom, i.e., from the negative z-direction to this object. A typical application is a station on which a *MU* is carried on its center of gravity, for example most of the length-oriented objects.



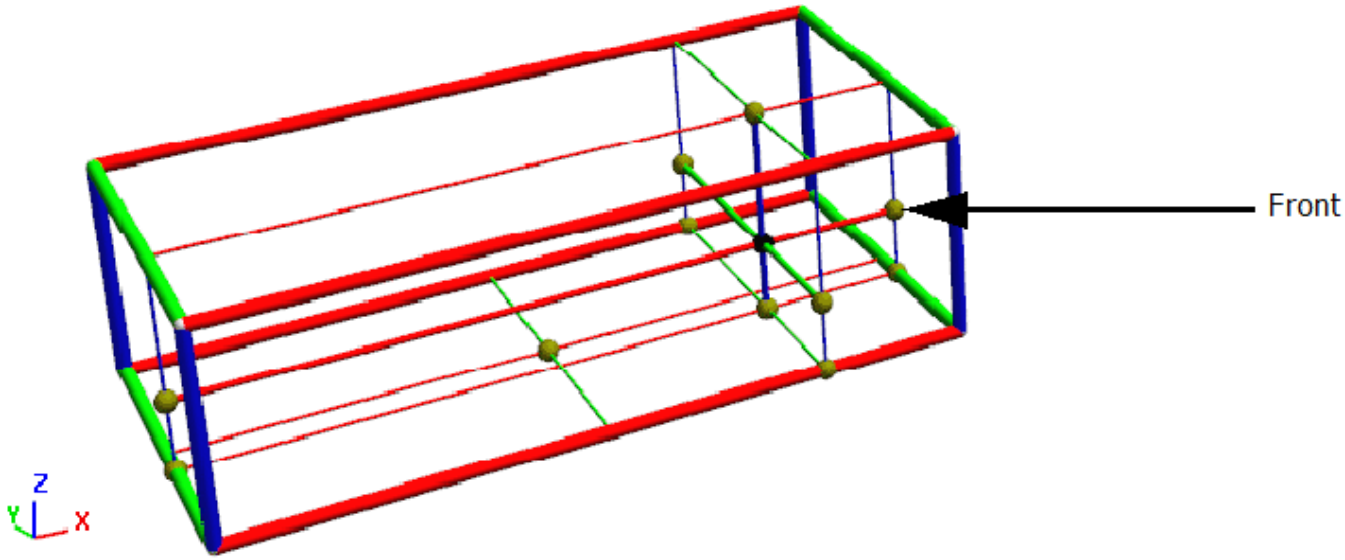
- `"Booking point"`: Attaches the loaded *MU* to the object from the local position (**0**, **0**, **0**) to this object. The local position is the position which you set on the tab **Attributes** in the dialog of the *MU*.



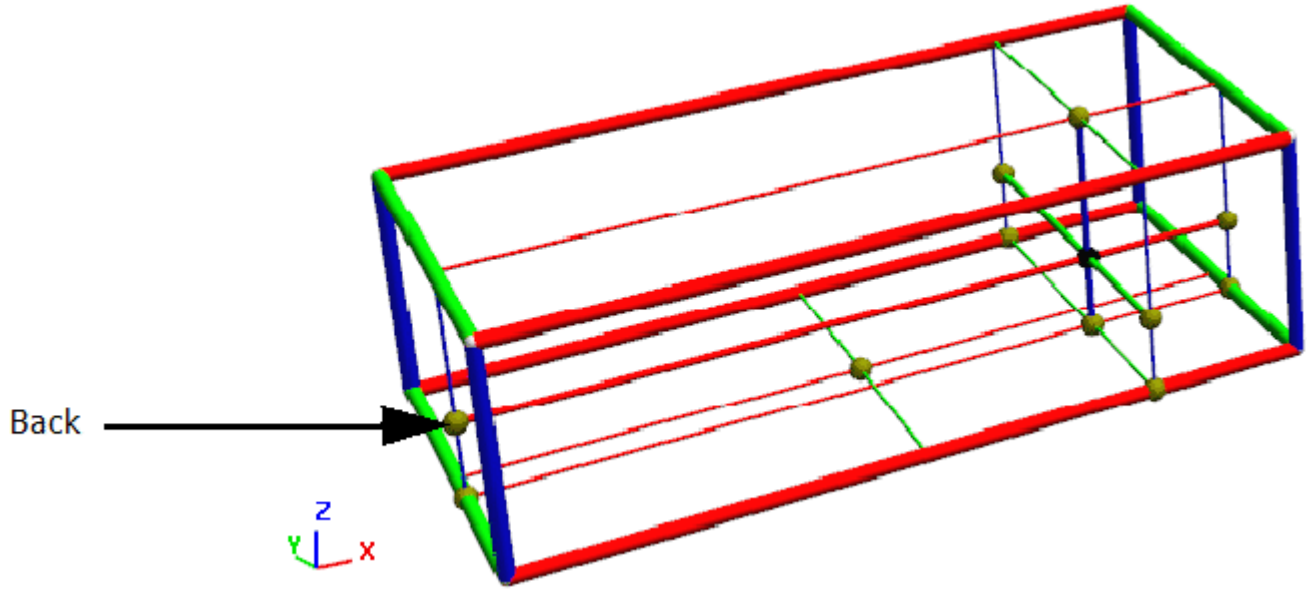
- `"Top"`: Attaches the loaded *MU* to this object from the top, i.e., from the positive z-direction. A typical application is a station on which a *MU* is attached on its center of gravity, for example an overhead rail system.



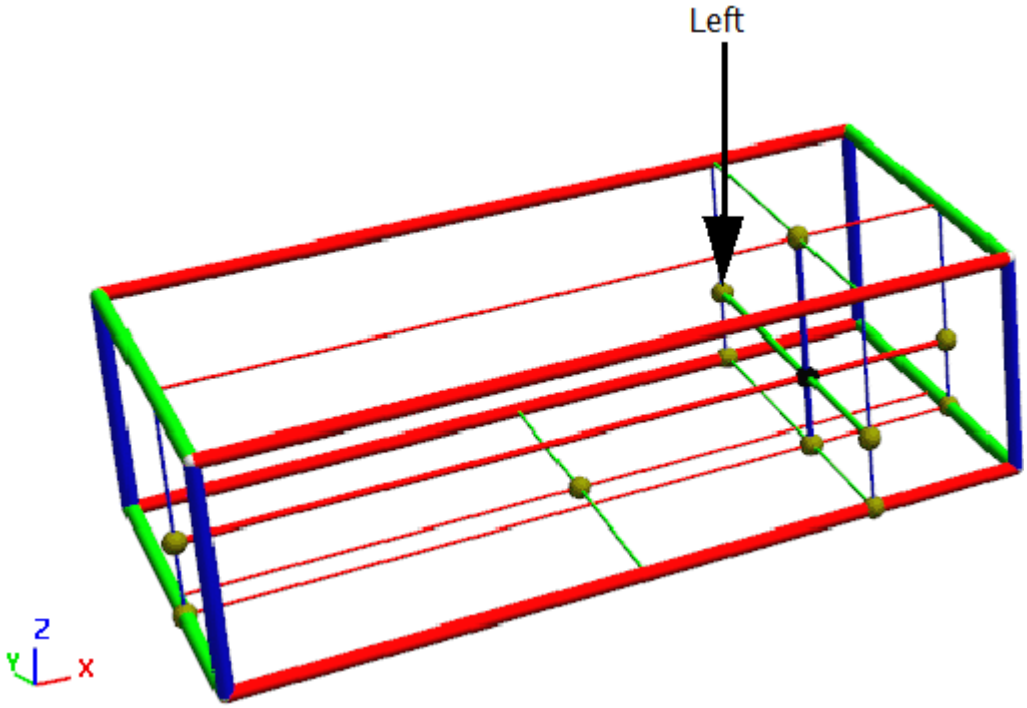
- `"Front"`: Attaches the loaded *MU* to this object from the front, i.e., from the positive x-direction.



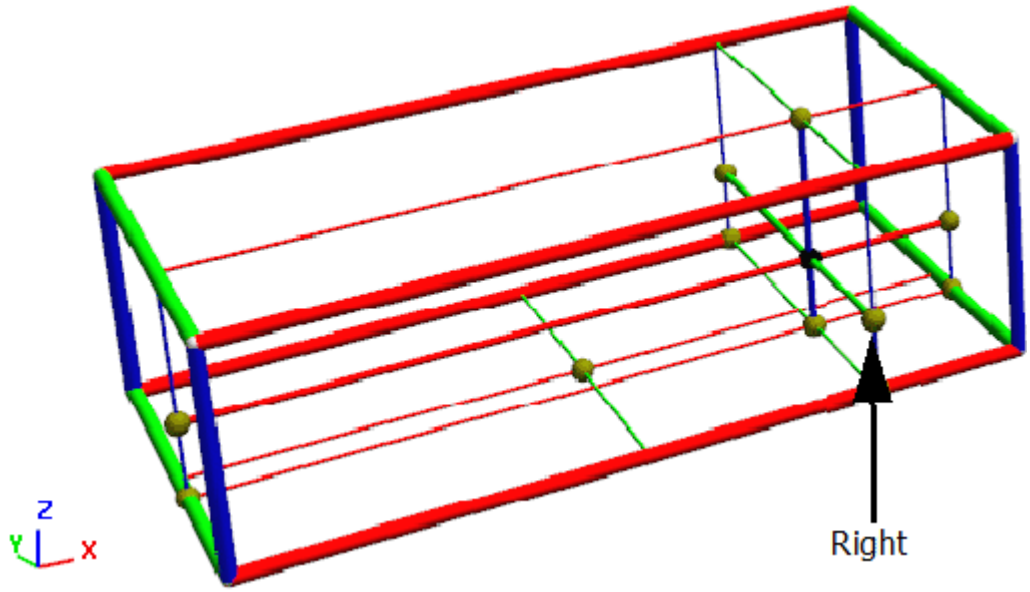
- `"Back"`: Attaches the loaded *MU* to this object from the back, i. e., from the negative x-direction.



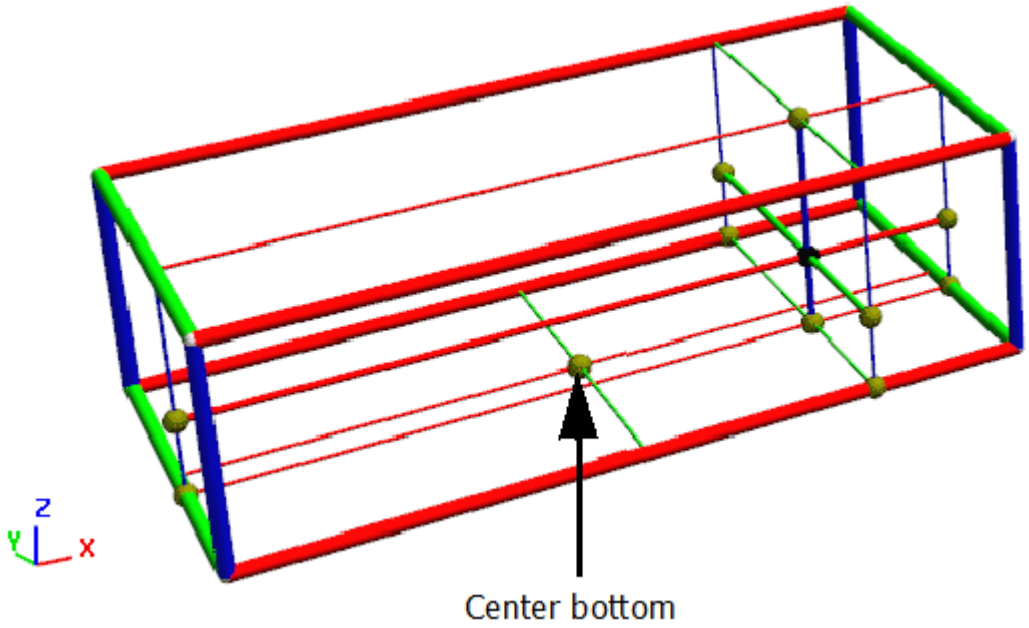
- `"Left"`: Attaches the loaded *MU* to this object from the left-hand side, i.e., from the negative y-direction.



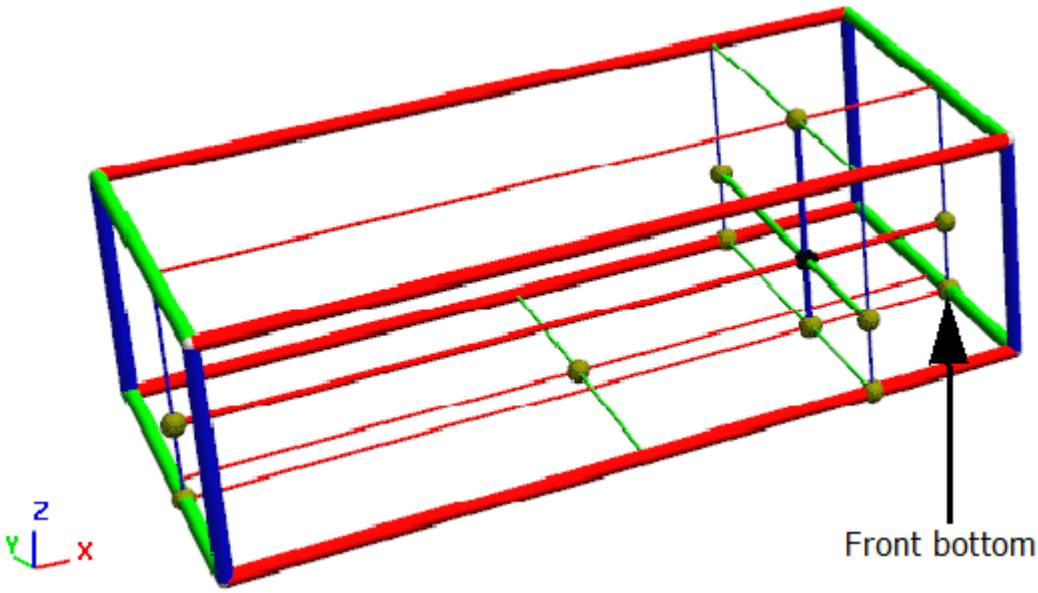
- `"Right"`: Attaches the loaded *MU* to this object from the right-hand side, i.e., from the positive y-direction.



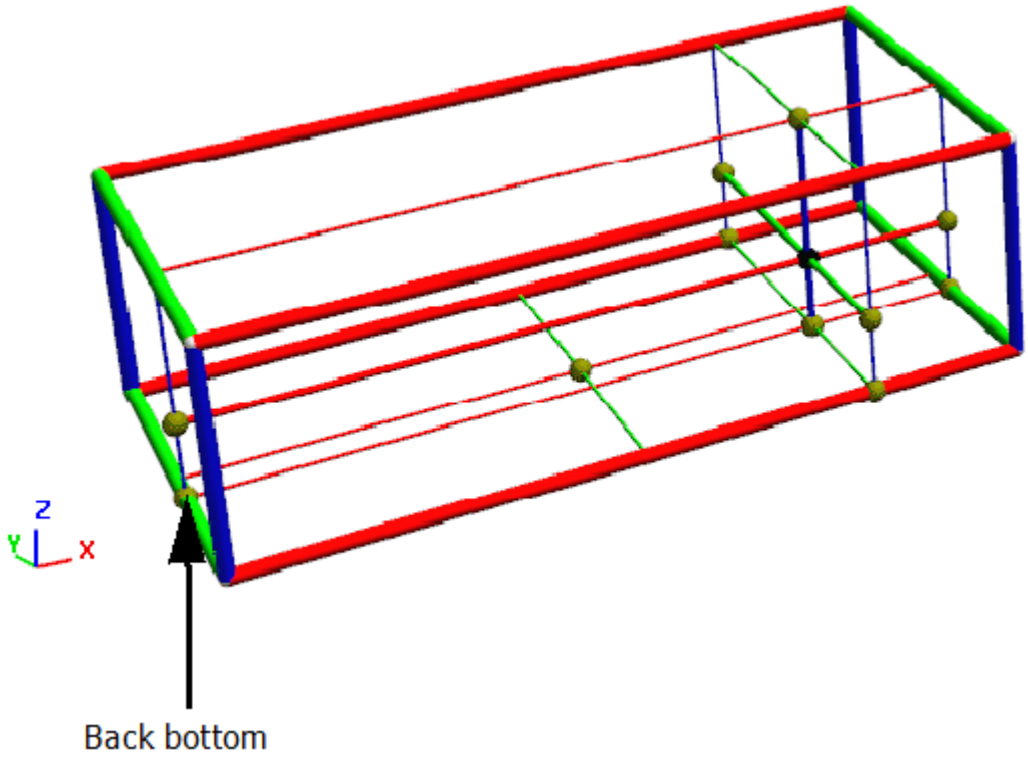
- `"Center bottom"`: Places the loaded *MU* with the middle of the base plate of its dimensions onto the animation path of the transporting object. A typical application is a station on which a *MU* is placed planar.



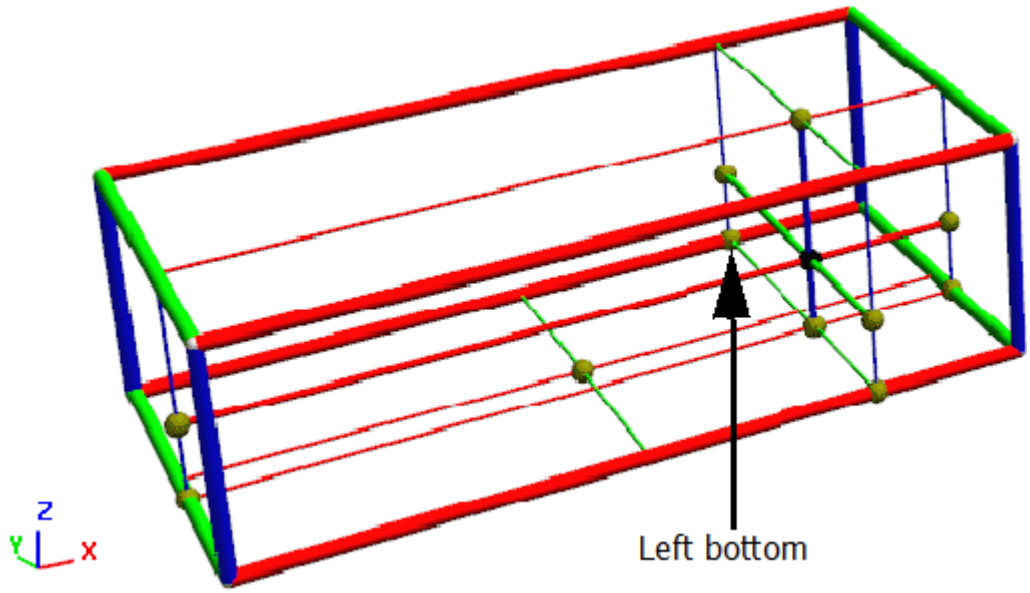
- `"Front bottom"`: Combines the two settings **Front** and **Bottom**.



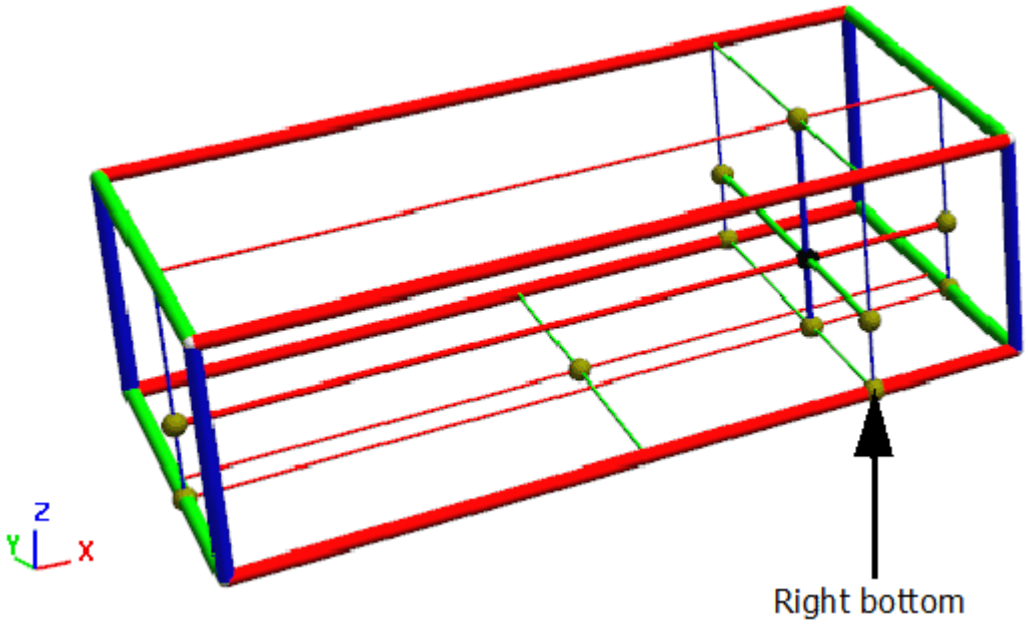
- `"Back bottom"`: Combines the two settings **Back** and **Bottom**.



- `"Left bottom"`: Combines the two settings **Left** and **Bottom**.



- `"Right bottom"`: Combines the two settings **Right** and **Bottom**.



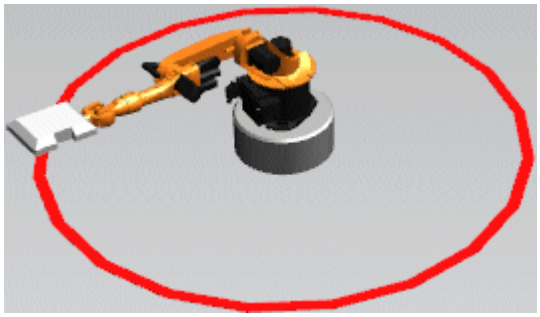
For the **side on which the part is attached to the object**, *Plant Simulation* takes the following into account:

- The **Conveying Direction [described]** of the *MU*. This means, for example, that the setting **MU Side To Attach > Left** does not in any case attach the left side of the *MU* from the standpoint of the *MU* to an animation line, but onto the back for the **Conveying Direction > 1 (lateral right)**.
- The **Booking Point Height [general description]** of the *MU*.
- The **MU Length [general description]**, the **MU Width [general description]**, and the **MU Height [general description]** which is entered on the tab **Attributes** of the *MU*.

For the **PickAndPlace-Robot** and its animatable objects the following applies:

> **Note**
> The **MU Side To Attach** also changes the orientation of the loaded part as appropriate.

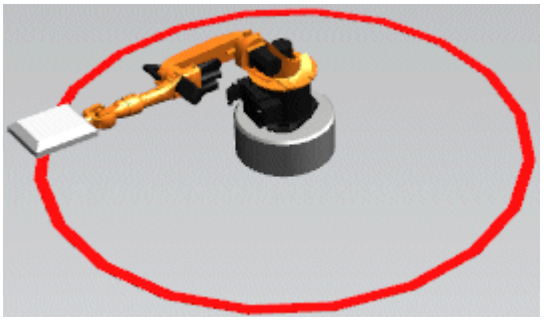
- `"Front"`: *Plant Simulation* applies a rotation of 90° around the y-axis, followed by a rotation of 180° around the z-axis.



- `"Right"`: *Plant Simulation* applies a rotation of -90° around the x-axis.

- `"Left"`: *Plant Simulation* applies a rotation of 90° around the x-axis.

- `"Back"`: *Plant Simulation* applies a rotation of 90° around the y-axis.



Thereby these four settings and the setting **Top** should point to the respective face normals for an unrotated animation <Path>.

The settings for **Bottom** apply the same orientation as **Top**, i.e. the orientation that is used by all other objects as well.

Example

```
Station._3D.MUSideToAttach := "Left"
```

SimTalk

rotateConveyingDirection [SimTalk]

See also

Edit 3D Properties [dialog] > **MU Animation [tab]** > **MU Side to Attach**

Conveying Direction [described]

_3D.resetAnimationTime [SimTalk]

Resets the animation time of a *MU* on the object designated by <Path> on which it is located to 0.

Remarks

A time-controlled **MU Animation** always starts when the *MU* moves onto the object on which it will be processed or transported.

Use the method *_3D.resetAnimationTime* to start an animation from 0 at a later point in time. If you do not do this, the animation to be used will be changed, but does not start at 0, but with an initial time offset, which corresponds to the time which the *MU* is already located on the current object.

> **Note**
> The *Worker* does not support the method *_3D.resetAnimationTime*.

Type

Method

Syntax

```
<Path>._3D.resetAnimationTime
```

Example

```
@._3D.resetAnimationTime
```

MUAnimations.copyFromSimulationObject [SimTalk]

Copies all **MU Animations** from the simulation object that owns the animatable object designated by <Path> to that animatable object.

Remarks

If the simulation object does have any **MU Animation Paths**, the method copies these and overwrites all animation paths which the animatable object had beforehand, including those with names which the simulation object did not have.

If the simulation object (and thus the animatable object as well) can animate *MUs* using a **MU Animation Area** in principle, even if not currently used, the method copies the **Animation Area** (including the setting that specifies whether to use it) from the simulation object to the animatable object.